Super Tic-Tac-Toe Reinforcement Learning Report

1. Background

Super Tic-Tac-Toe is a two-player adversarial environment with stochastic action perturbations. Compared with standard Tic-Tac-Toe, it has a larger action space, more complex win conditions, and random transitions, making it a good candidate for reinforcement learning.

The goal of this project is to train an agent that can still make stable decisions under random disturbances. To this end, Double DQN is adopted as the main algorithm, together with action masking, experience replay, and target network updates, in order to improve training stability and sample efficiency.

2. Method and Environment Design

2.1 State and Action Spaces

The state space is represented as a $6 \times 4 \times 4$ board tensor, where each cell indicates empty, current-player piece, or opponent piece. The action space contains 96 discrete actions, each corresponding to one specific board cell, and the agent must select an intended landing position at every step.

2.2 Reward Design

The reward signal is intentionally sparse: a win gives positive reward, while invalid moves, forfeited moves, and draws do not add extra reward. A light reward shaping term is also added during training to encourage the agent to extend existing lines and reduce the difficulty caused by sparse rewards.

2.3 Stochastic Perturbation

To simulate uncertainty, the environment perturbs the intended action with a certain probability. This encourages the agent to learn robust strategies rather than overfitting to a single exact landing point, and it better matches the stochastic transition design of the task.

2.4 Winning Rules and the Global Coordinate System

The board is arranged using a global coordinate system instead of judging only inside local 4×4 sub-boards. This makes it easier to detect cross-region and cross-level lines, and also keeps the web demo and test scripts consistent. In the environment design, this layout makes win detection more direct and unified.

The winning rules are defined as: 4 in a row horizontally; 4 in a row vertically with cross-level coverage; and 5 in a row diagonally.

Note: the level rule is implemented according to the global coordinate system, so vertical 4-in-a-row wins depend not only on continuity but also on whether the line spans multiple level regions.

Chessboard Layout

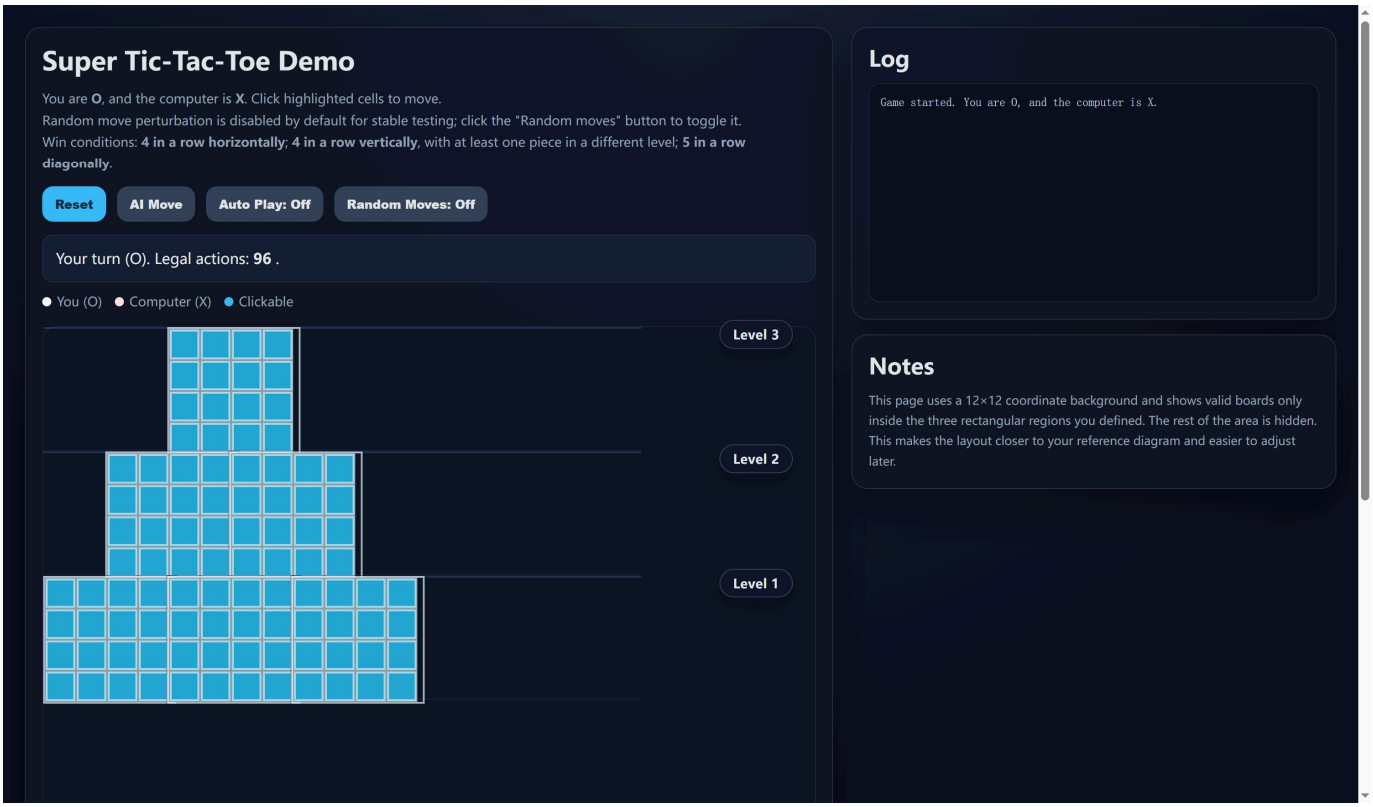

Figure 1: Global coordinate layout of the board. Using a global coordinate system makes win detection direct and consistent.

2.5 Why Use a Global Coordinate System

The main reason is to simplify win detection. Because the level structure is distributed across multiple 4 × 4 boards, a local-only coordinate check can easily miss lines that extend across regions. A global coordinate system maps all cells onto a single board, making horizontal, vertical, and diagonal line checks straightforward and less ambiguous. With this design, one consistent coordinate rule can be used for move logging, win checking, and visualization.

2.6 Random Move Mechanism

To satisfy the assignment requirements, a random move mechanism is kept in the web demo. When this feature is enabled, the selected move may be shifted to a neighboring cell before being applied. This mirrors the stochastic transition used in the reinforcement learning environment and makes the demo closer to the actual training setting while still preserving visibility of the randomness effect.

Random Move Demo

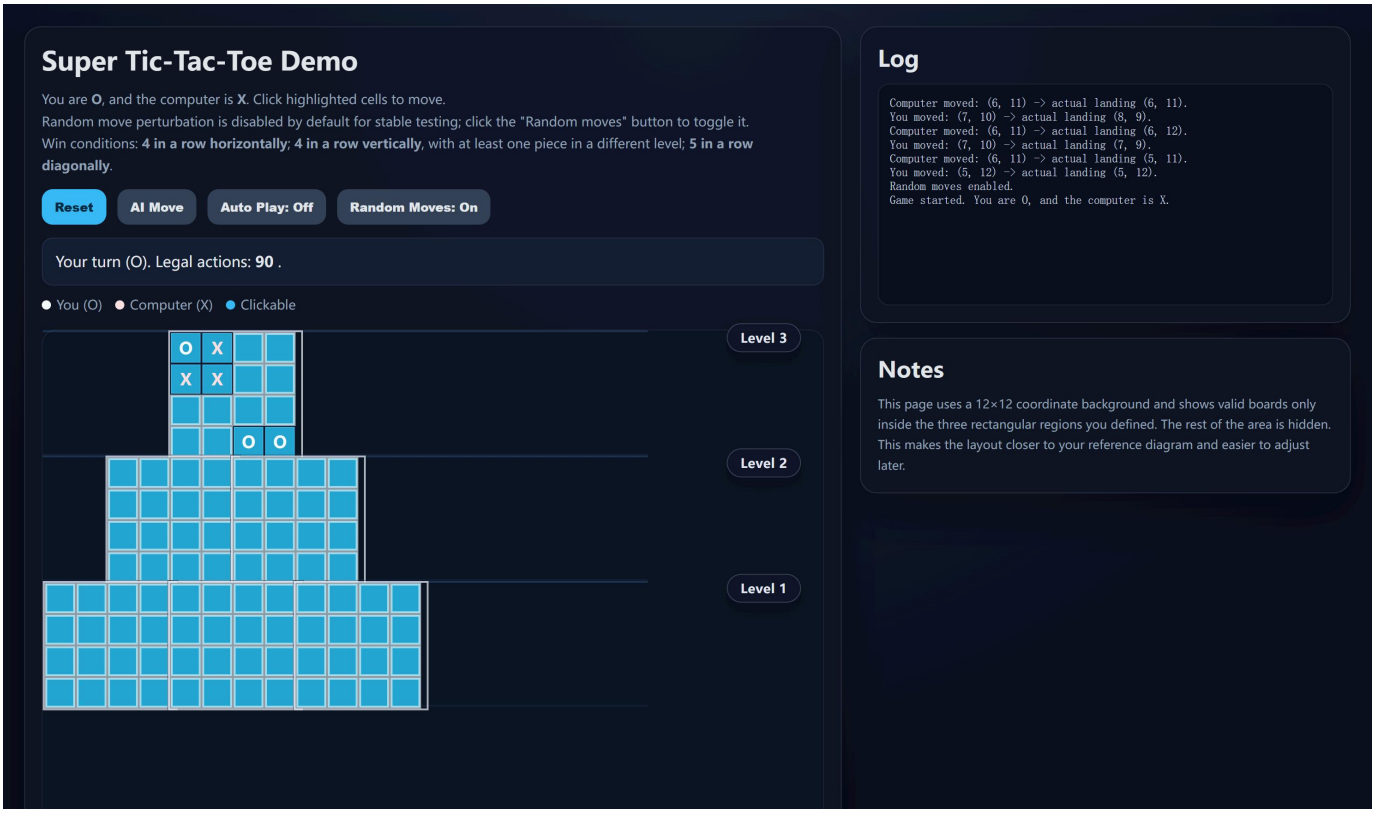

Figure 2: Random move demonstration. When enabled, the actual landing position may shift to a neighboring cell.

3. Algorithm: Double DQN

3.1 Why Double DQN

This task is a discrete-action adversarial game, so DQN-style methods are a natural choice. Compared with vanilla DQN, Double DQN reduces Q-value overestimation and is usually more stable in stochastic and adversarial settings.

3.2 Core Idea of Double DQN

Double DQN uses two networks: the online network selects the next action, while the target network evaluates the value of that action. By separating action selection from action evaluation, the max-operator overestimation in standard DQN is reduced.

3.3 Experience Replay, Target Network, and Action Masking

Experience replay breaks the correlation between consecutive transitions and improves training stability. The target network smooths target estimation and reduces oscillation. Action masking filters out illegal moves so the agent only explores valid actions.

Together, these components make Double DQN a strong fit for this stochastic adversarial environment.

4. Training Curve Analysis

Model Training Curve

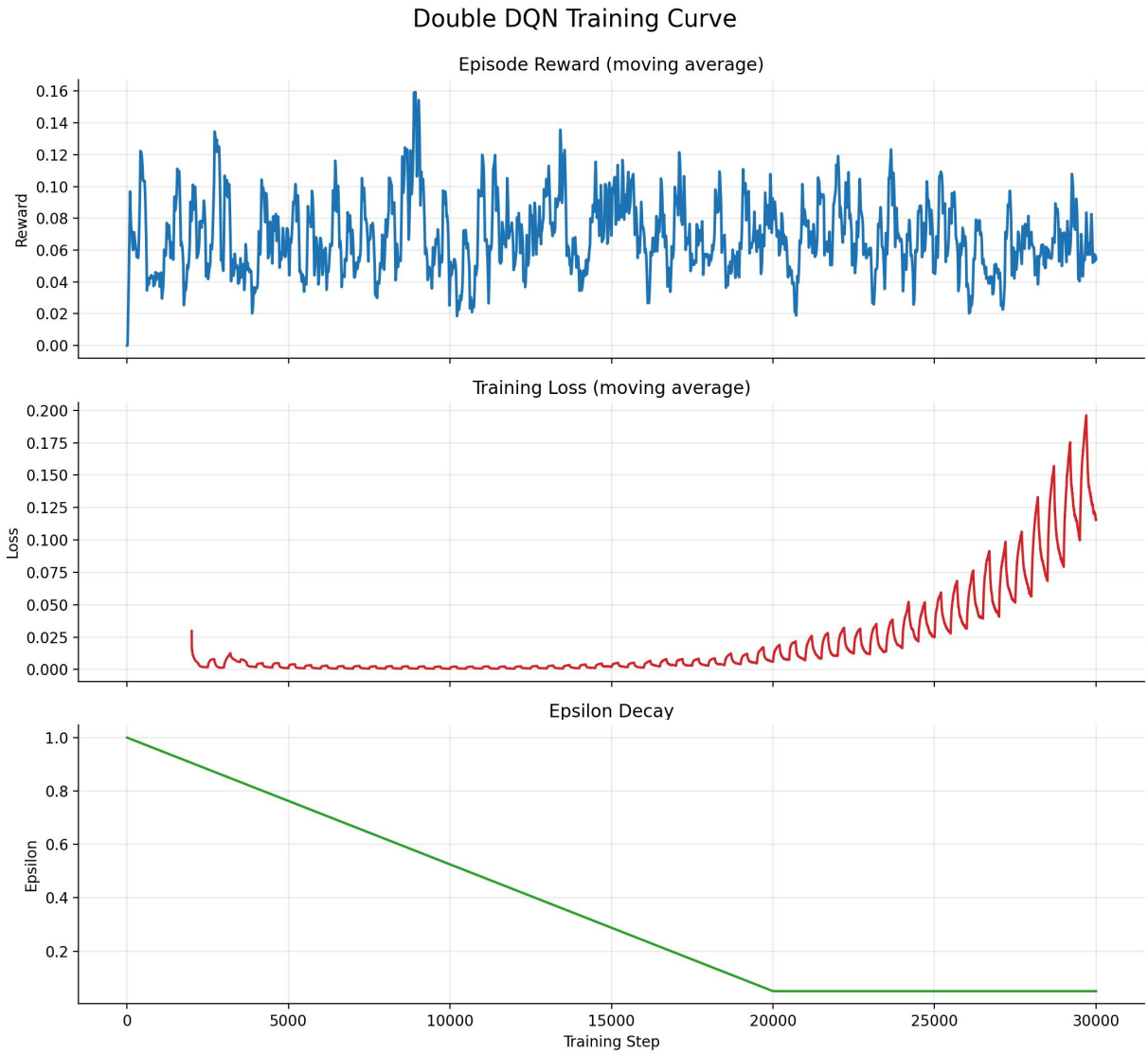

Figure 3: Training reward, loss, and epsilon decay recorded during training.

The training curve shows that epsilon gradually decreases as training progresses, which means the agent moves from random exploration toward policy exploitation. At the same time, the loss curve generally fluctuates at the beginning and becomes more stable later, indicating that parameter updates are gradually converging and the optimization process is settling down.

The reward curve is more volatile in the early stage because the agent is still exploring and the environment includes random perturbations. As more useful transitions are collected in the replay buffer, the reward trend becomes more stable and begins to rise, showing that the agent is learning feasible move strategies and obtaining more stable returns in competitive play.

Overall, the training curve reflects the optimization process of Double DQN and also suggests that the environment design, action masking, and reward structure are sufficient to support continuous learning and steady policy improvement.

5. Gameplay Figures

5.1 Horizontal Win Demo

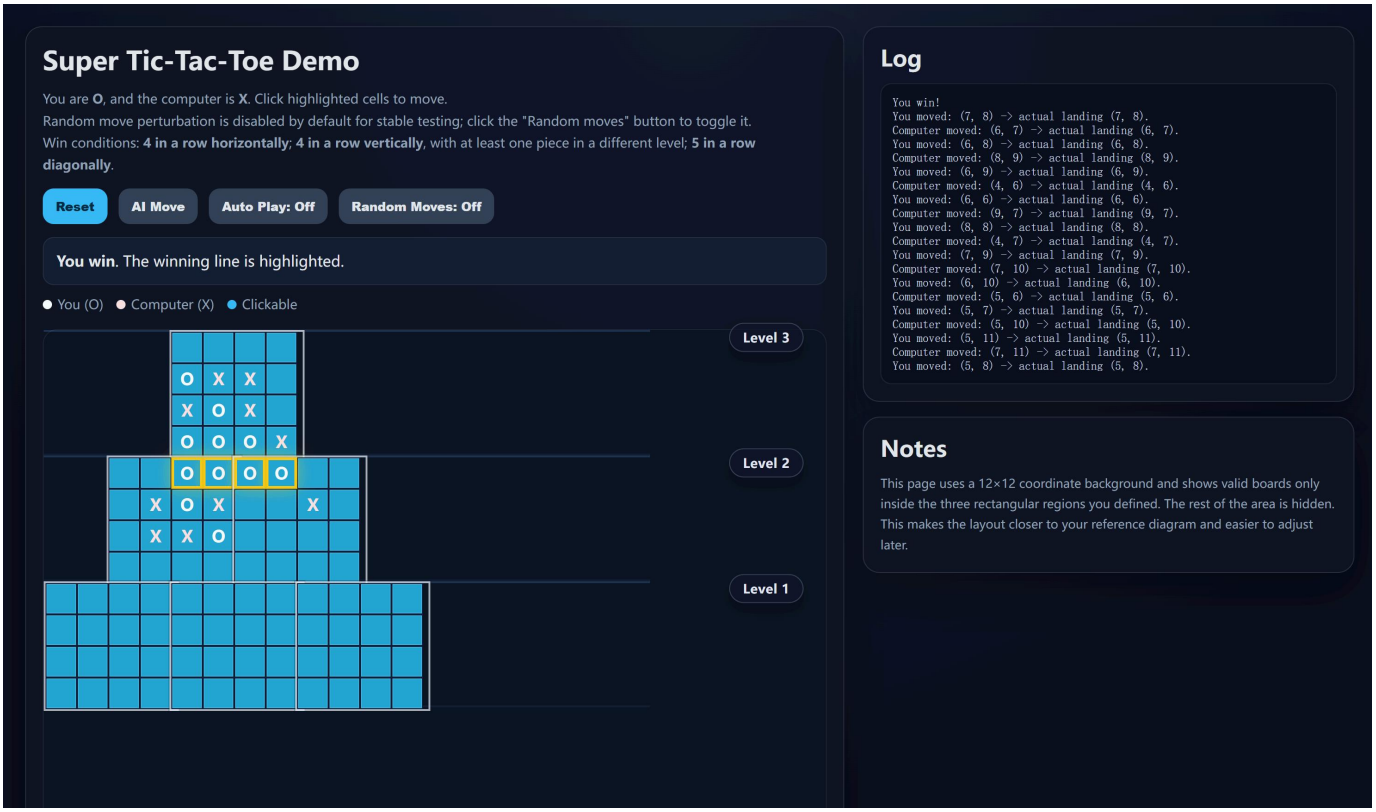

Figure 4: A 4-in-a-row horizontal win.

5.2 Vertical Cross-Level Win Demo

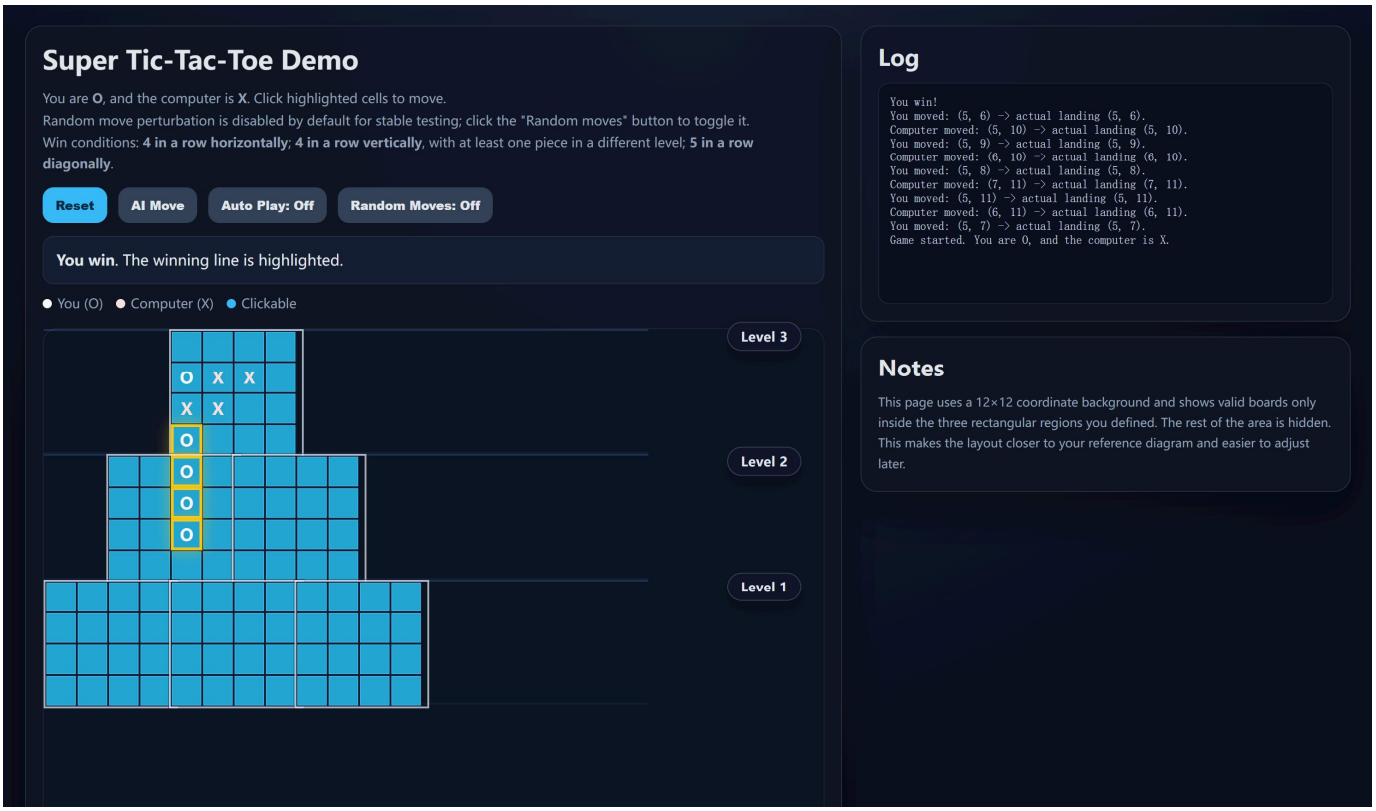

Figure 5: A 4-in-a-row vertical win spanning multiple levels.

5.3 Diagonal Win Demo

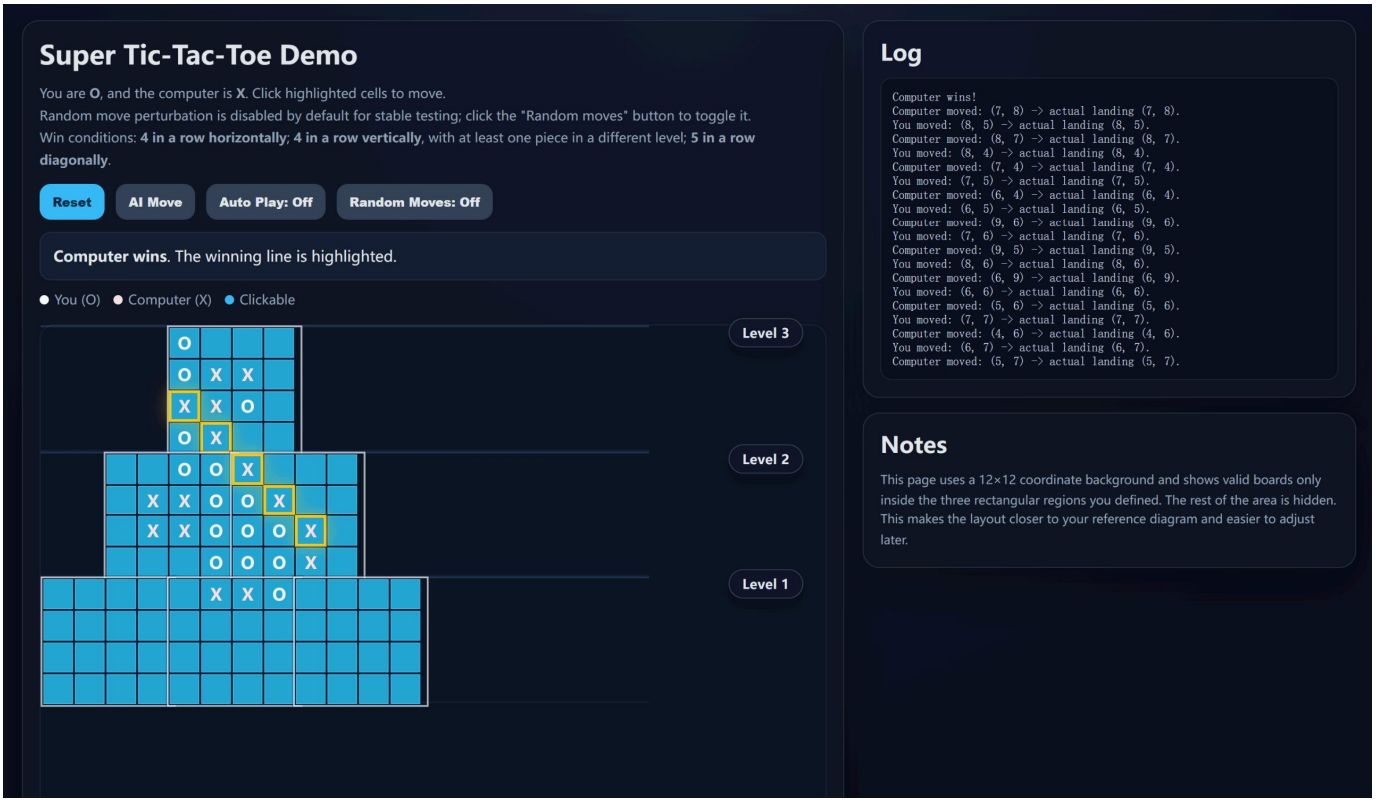

Figure 6: A 5-in-a-row diagonal win.

5.4 AI Blocking Demo

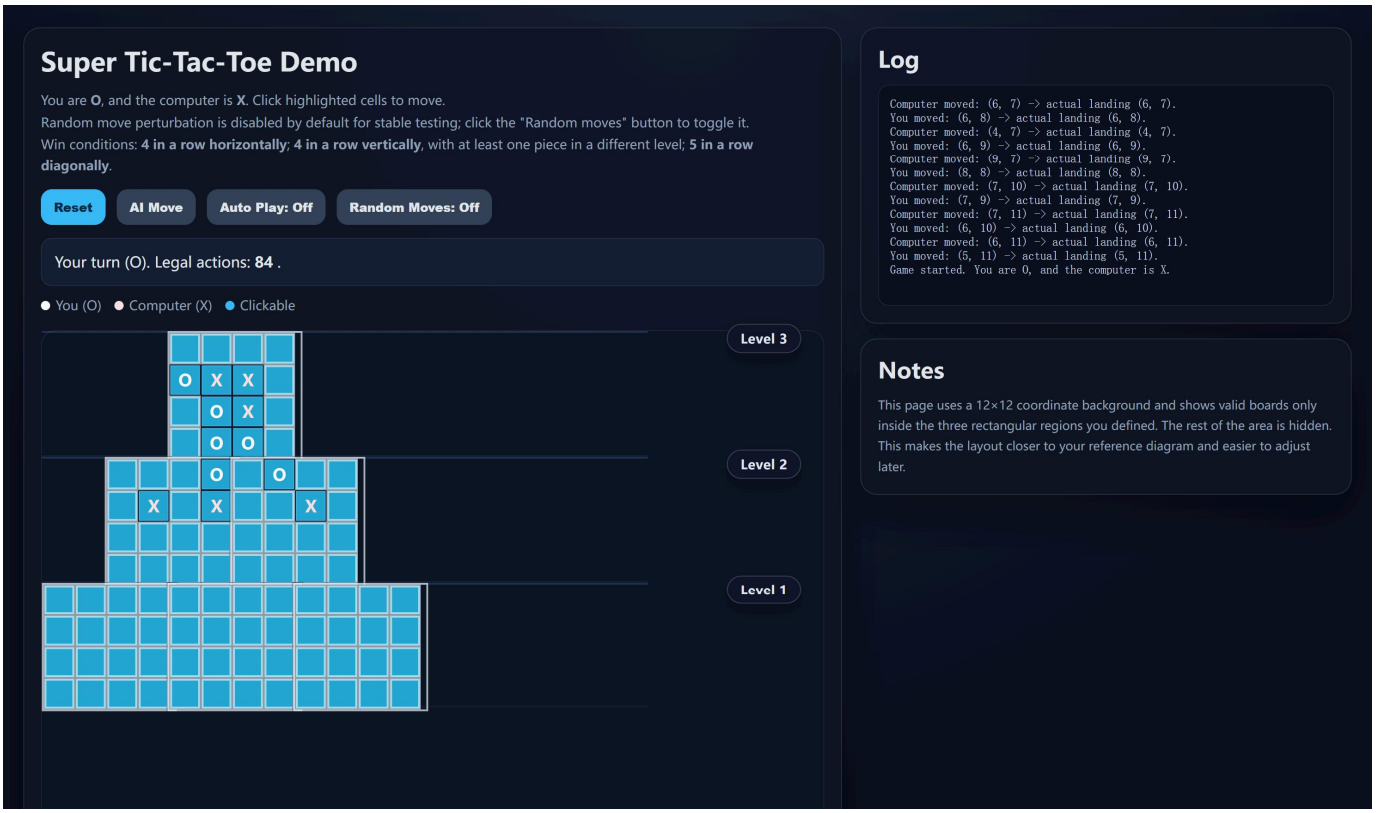

Figure 7: The AI identifies and blocks a potential winning threat.

6. Analysis

In the early stage of training, the agent relies heavily on random exploration, so action quality is limited. As experience accumulates, the agent gradually learns basic attacking and defensive patterns.

After introducing action masking, invalid exploration is greatly reduced. Combined with the target network in Double DQN, the training process becomes more stable than vanilla DQN, and the Q-value estimates become smoother.

The gameplay figures show that the agent can recognize different winning patterns and also intercept threats from the opponent, indicating that it has learned meaningful adversarial behavior and can react to changes in the board state.

The global coordinate system further improves rule consistency: horizontal, vertical, and diagonal lines can all be checked directly on the same board space, avoiding ambiguity between local sub-boards and keeping the implementation easier to verify.

7. Conclusion

This project successfully models Super Tic-Tac-Toe as a reinforcement learning problem and trains a usable policy with Double DQN. The training curve and gameplay screenshots verify that the agent has learned basic line formation, win recognition, and defensive blocking skills.

Overall, the global coordinate system makes the board layout easier to understand and the win conditions easier to implement, while also keeping the environment, web demo, and test scripts aligned.